GSoC 2026 — INCF Project #33

AStats: Agentic AI for Applied Statistical Practitioner Workflows

A Human-in-the-Loop Approach to Trustworthy Statistical Analysis

1. Contact Details

Full Name	Shreenidhi Gopalakrishnan
Email	shreenig99@gmail.com
Neurostars Username	Shreenidhi_G
Location	Coimbatore, Tamil Nadu, India
GitHub (AStats prototype)	https://github.com/ShreeniG99/AStats-Agentic-AI-for-Applied-Statistical-Practitioner-Workflows
Institution	Amrita Vishwa Vidyapeetham — B.Tech AI & Data Science (4th Semester), Minor in Medical Engineering
Mentors	Jonathan Morris (UW-Madison), Yohai-Eliel Berreby (McGill), Suresh Krishna (McGill)
Project Size	Full-time — 350 hours

2. Project Synopsis

AStats is a new, greenfield project. The goal is to build an agentic AI system that helps researchers carry out statistical analysis in a way that is both correct and transparent. The system will take a plain language question about a dataset, figure out the right statistical test to use, run it, and explain the result. But unlike existing tools, it keeps the researcher in control at every step — it shows its reasoning, waits for confirmation, accepts corrections, and saves every decision for reproducibility.

My specific focus is building the complete end-to-end pipeline: from the user's question in English, through data structure detection and assumption checking, to test selection, execution, and explanation — all wrapped in a feedback loop that the practitioner controls.

3. Project in Detail

What is the actual problem?

Picking the right statistical test should be straightforward, but in practice researchers spend a lot of time getting it wrong. There are three specific failure modes that happen repeatedly:

- Current LLMs jump to naming a test without checking the data first. I ran a benchmark comparing GPT-4.1 against a structured pipeline on 13 test scenarios — GPT-4.1 passed with a 45%, compared to the structured approach that I took which passed with 69%. The main reason for GPT-4.1's failures: it never checked whether the data was normally distributed, and it confused independent groups with repeated measurements.
- Tools like JASP and Jamovi are standard and useful, but they don't understand natural language questions, can't adapt to context, and have no way for the researcher to push back on a result.

- The most statistically dangerous mistake is pseudo-replication — treating repeated measurements on the same person as if they're independent observations. This can increase sample size by 10x or more, and most automated tools have no check for it.

My proposed system handles all three gaps: understanding the question, detecting the data structure, and keeping the researcher in control of the final call.

What I will build

The system has five parts that work together as one pipeline:

- **1.** An NL query parser that reads a plain-English question, detects whether the user wants to compare groups, find a correlation, or predict an outcome, flags ambiguous queries before going further, and extracts hints about which columns are the outcome and the grouping variable.
- **2.** A data structure inference engine that looks at the actual dataset and decides whether it is independent groups, repeated measures, nested, or wide-format repeated measures. The repeated measures detection specifically checks observation count per subject and minimum observations to avoid falsely flagging a patient ID column as a repeated measures design.
- **3.** An assumption checking layer that runs Shapiro-Wilk for normality, Levene's test for variance equality, and a sample adequacy check. For repeated measures data, it correctly computes normality on within-subject difference scores rather than raw values.
- **4.** A test selection and execution engine with a deterministic decision matrix covering 9 tests. It runs the chosen test, computes an appropriate effect size for every test (Cohen's d, rank-biserial r, Kendall's W, eta-squared), and generates a methods paragraph.
- **5.** A practitioner feedback loop- Before finalising any result, AStats shows the recommendation and reasoning. The researcher can accept, override with any of 9 tests and give a reason, ask for a plain-language explanation of why this test was chosen, or correct an assumption. Every interaction is saved to a JSON session file with timestamps.

The practitioner is always in control. The --auto flag skips the interactive prompts for non-interactive use, and all output modes (rich terminal, plain text, JSON, methods-only) are supported.

Why the feedback loop is the key contribution

This is the part that no other AStats prototype has built. I came to think about this through my own work. During my second year, I built a clinical decision support tool for adverse drug reaction prediction using a LightGBM model trained on FDA FAERS pharmacovigilance data. However, I had no mechanism to capture corrections and route it back into the model, or even record it formally.

I realized that a statistical recommendation without a correction mechanism is not enough. The session logging creates something more useful: a growing record of corrections that can improve the tool over time. This is the design philosophy behind the whole project.

Proof of concept — what I have already built

I have built a working end-to-end prototype before submitting this proposal. Here is what it does, in comparison to existing prototypes created by Utkarsh and Atta- whose projects I referred to from the github repo link that they had included in their posts:

Feature	Utkarsh's PoC	Atta's PoC	This PoC
NL query parsing	No	Yes	Yes
Data structure inference	Yes	No	Yes
Assumption checking	Yes	No	Yes
Test execution + effect size	Yes	No	Yes

Methods paragraph output	Yes	No	Yes
Practitioner feedback loop	No	No	Yes
Session logging to JSON	No	No	Yes
Plain-language explanation	No	No	Yes
Multiple output modes (JSON/plain/methods)	No	No	Yes

The benchmark runs 13 scenarios including all three known failure cases from Utkarsh's prototype — small sample routing, borderline repeated measures, and non-normal within-subject differences. The PoC passes 9/13 (69%). The 4 failures are documented statistical judgment calls where even experienced statisticians disagree.

I also built 8 clinically grounded test datasets covering every analysis path in the pipeline:

1. A 93-patient antidepressant RCT (independent t-test)
2. A 20-subject sleep deprivation study modelled on Belenky et al. (Friedman)
3. A stroke rehabilitation pre/post study (paired t-test)
4. A 4-arm pain trial (ANOVA)
5. A skewed reaction time data (Kruskal-Wallis)
6. A wide-format EEG alpha power across 5 brain regions (Friedman wide)
7. A CRP-cognitive decline correlation study, and
8. A 155-patient Hepatitis B ADR study mirroring my own FAERS work.

AStats Prototype (ver1): Link to my github repo

Technical choices

Python with Click for the CLI and Rich for terminal output. Statistical tests from SciPy and Statsmodels. LLM explanation uses the Anthropic API (key is not available right now), with a built-in fallback covering all 9 tests in detail — so no API key required to run the tool. Both paths are transparent to the user.

R integration is planned for week 5 using rpy2, covering tests where R is clearly more capable: lme4 for mixed models, BayesFactor for Bayesian t-tests, and car for Welch ANOVA. Python handles the NL parsing, structure inference, and session logic; R handles the statistical heavy lifting where needed.

For data auto-discovery, I will extend ydata-profiling with structure-aware analysis specific to AStats — detecting repeated measures patterns, flagging pseudo-replication risk, and suggesting relevant analyses before the user asks a question.

4. Deliverables

Minimum deliverables (guaranteed)

Core agent architecture

- AStats implemented as a LangGraph state machine with a defined agent loop: Perceive → Plan → Analyse → Reflect → Report. The agent decides which tools to call and in what order, rather than executing a fixed pipeline. This is the architectural difference between a smart CLI and a genuine agent.
- A tool registry of 12+ callable tools: load_data, infer_structure, check_assumptions, select_test, run_test_python, run_test_r, compute_effect_size, explain_decision, detect_outliers,

run_posthoc, generate_methods_para, log_correction. The agent selects from these based on context.
- Self-correction loop: if the agent's initial test selection is inconsistent with the data (e.g. normality check contradicts the chosen test), it backtracks and reruns before presenting to the practitioner.

Analysis capabilities

- Full statistical pipeline covering 9 tests: Independent t-test, Welch's t-test, Mann-Whitney U, Paired t-test, Wilcoxon signed-rank, Friedman, One-way ANOVA, Welch's ANOVA, Kruskal-Wallis H. Each with effect size and a copy-paste ready methods paragraph.
- Data structure inference: independent, repeated-measures, nested, wide-format.
- Assumption checking: Shapiro-Wilk, Levene's, sample adequacy, correct within-subject difference logic for repeated measures.
- Post-hoc tests after significant ANOVA or Kruskal-Wallis: Tukey HSD, Bonferroni correction, Dunn's test — triggered automatically by the agent, not by the user asking.
- Outlier detection flagged before test selection, with agent noting potential impact on results.
- R bridge via rpy2 for 3 R-native tests: lme4 mixed models, BayesFactor Bayesian t-test, car Welch ANOVA.

Human-in-the-loop system

- Multi-turn session: the practitioner can ask follow-up questions in the same session without reloading the dataset ("now try Mann-Whitney instead", "what if I remove the outlier in row 5", "run the same analysis on the female subgroup only"). The agent maintains context across turns.
- Full interaction menu: accept, override, explain (rich built-in explanation for all 9 tests, works offline), fix assumption, ask follow-up question.
- Session logging to JSON: every decision, every override, every correction, every follow-up — timestamped and saved with reasons. Designed as DPO-compatible preference data from day one.
- All output modes: rich terminal, plain text, JSON (machine-readable), methods-only (copy-paste for paper).

Infrastructure

- Benchmark harness: 20+ labelled scenarios including all known failure cases from prior prototypes, plus 8 real clinical/neuroscience datasets (antidepressant RCT, sleep deprivation longitudinal, cognitive rehabilitation pre/post, 4-arm pain trial, skewed reaction time data, wide-format EEG alpha power, CRP-cognitive decline correlation, Hepatitis B ADR study).
- Full documentation: README, getting-started guide, API reference, tutorial notebooks.

User Interface

- Web interface using Next.js with shadcn/ui, connected to the LangGraph agent via a FastAPI streaming backend. The UI streams agent reasoning steps in real time as they happen — structure detection, assumption checks, test selection — giving the practitioner live visibility into the agent's thought process before the final recommendation appears. This mirrors the interaction model of modern agentic AI interfaces and makes the human-in-the-loop design visually tangible rather than just a CLI menu.

Stretch goals (if time allows)

DPO fine-tuning from session corrections The session correction log captures structured preference data — which test AStats recommended, which the practitioner chose instead, and why. I will format these as (rejected, accepted) pairs and apply Direct Preference Optimization (DPO) to fine-tune Mistral 7B via Ollama. DPO achieves similar outcomes to RLHF without needing a reward model or RL training loop, making it feasible on consumer hardware within the project scope. This is contingent on collecting sufficient correction volume.

Autonomous exploratory mode Given a raw dataset with no query, the agent autonomously explores it — detects structure, identifies interesting variables, runs a suite of relevant analyses, and produces a structured exploration report. No question needed from the user. This is the most "agentic" possible expression of AStats.

Natural language follow-up across sessions "Last week you analysed the antidepressant data and found p=0.001 — run the same analysis on the new dataset." Cross-session memory using the correction log as a persistent knowledge base.

5. Timeline

Period	Focus	Concrete Tasks
May 8 – Jun 1	Architecture design	Sync with mentors. Design the LangGraph state machine on paper — define all nodes, edges, tool interfaces. Study the AStats community on alphatest.scicommons.org. Reproduce sleepstudy validation independently. Decide on R bridge strategy with mentors.
Weeks 1–2 (Jun 2–15)	Agent skeleton	Refactor PoC into LangGraph-based agent. Implement tool registry with all 12 tools as stubs, then fill in each one. Unit tests for every tool. All existing benchmark scenarios passing.

Weeks 3–4 (Jun 16–29)	Agent intelligence	Add self-correction loop (agent backtracks on inconsistency). Implement post-hoc tests triggered automatically after ANOVA/Kruskal-Wallis. Improve NL parser with LLM-backed intent detection. Multi-turn session support.
Weeks 5–6 (Jun 30 – Jul 13)	R bridge + robustness	rpy2 integration: lme4, BayesFactor, Welch ANOVA via car. Round-trip Python→R→Python testing. Add 10 more benchmark scenarios. Outlier detection module.
Week 7 (Jul 14–20)	Midterm	Midterm evaluation submission. Document everything built so far. Address mentor feedback. Benchmark now at 30+ scenarios.
Weeks 8–9 (Jul 21 – Aug 3)	Autonomous exploration	Build the no-query exploratory mode. Agent analyses a raw dataset unprompted and produces a structured report. ydata-profiling as base, extended with structure-aware suggestions.
Weeks 10–11 (Aug 4–17)	Session tooling + DPO	Cross-turn session memory. Export session as a reproducible Python script. Format correction log as DPO preference pairs. Run first fine-tuning experiment on Mistral 7B via Ollama if correction volume allows.
Weeks 12–13 (Aug 18 – Sep 1)	Polish + submission	User testing on real research datasets from mentors if available. Full documentation, API reference, tutorial notebooks. Final code review. Submit.

6. Communication Plan

Weekly or biweekly Microsoft Teams meetup with mentors — whichever frequency they prefer. Questions and updates on the AStats community at alphatest.scicommons.org. I will push to GitHub as I make improvements to my work on a regular basis. Before any significant design decision — especially R bridge architecture or the fine-tuning direction — I will write a short note and share it before coding, so mentors can give their feedback and correct the plan early.

I am in IST (UTC+5:30), which overlaps well with North American afternoons. I respond to async messages within 24 hours during the programme.

7. About Me

Background

I am a 4th-semester B.Tech student in AI and Data Science at Amrita Vishwa Vidyapeetham, with a Minor in Medical Engineering. My work sits at the intersection of machine learning and real clinical or scientific data, which is exactly the territory AStats operates in.

Projects directly relevant to AStats

- Adverse Drug Reaction Prediction : LightGBM multi-output classifier trained on 8,000+ FDA FAERS records for Hepatitis B antivirals. Deployed as a Streamlit clinical decision support app with SHAP explainability. This is the project where I first confronted the practitioner feedback problem that AStats is designed to solve. The pharmacologist story in section 3 is real.
- BioChat AI : BioBERT embeddings, FAISS and Elasticsearch hybrid retrieval, LoRa fine-tuned BioGPT for biomedical literature question-answering. Directly relevant to the LLM explanation layer in AStats. Dockerized the app.
- Stage-Aware Career Co-Pilot (IETE AI-Verse hackathon): Multi-agent LangGraph orchestration system with tool-calling and state management. Direct experience with the agentic infrastructure AStats uses.
- OsteoPINN: Physics-informed neural networks for bone mineral density analysis. Demonstrates experience with domain-constrained deep learning for medical applications.
- SmartPosture (IEEE conference paper, 2024): ESP32 + dual MPU-6050 sensors + TensorFlow Lite for real-time on-device posture classification. Physiological time-series signal processing experience.

Why I want to do this project

The honest answer is that the AStats problem is one I've already hit personally and couldn't fully solve. When a professor told me a correction in my model’s output and I had no way to record it, that bothered me. AStats is the version of that problem where the correction loop is built in from the start. Also, the word “Agentic AI” captured my attention.

I also want to work on a greenfield project where architectural decisions actually matter. Adding a feature to an existing codebase is fine, but there is something different about starting from scratch with mentors who have a clear research vision. Sures sir's previous GSoC projects — from automated preferential looking to the APS eye-tracking GUI — all have this quality of solving a real scientific problem in a way that makes the tool genuinely usable by researchers who aren't software engineers. That is what I want to build here.

Why I am the right person for this

I have the three things this project needs working in combination: clinical data pipeline experience that makes the practitioner feedback problem real rather than theoretical, LLM and agentic AI experience from BioBERT/BioGPT and LangGraph, and enough statistical background to know where the automated pipeline should stop and ask a human. I have already built a working prototype before submitting this

proposal.

My R experience is basic, and my statistics goes to a solid intermediate level rather than deep theoretical. I will learn as the project requires it. Suresh has said explicitly in the thread that willingness to learn quickly is what matters, and that is consistent with how I work. I believe that I should be selected, because I'm a person who wants to prove myself. I will bring determination, persistence and results to the project.

Other applications

This is the only project I am applying for this year.

Time and availability

I can commit 35 hours per week for the full programme. My semester exams finish by April 1st week, before the coding period starts. I have no other jobs or internships planned for the summer. There may be a day or two of minor academic events in July — I will flag these to mentors well in advance.